Seminario de Arquitecturas de Software Distribuidas

Ingeniería de Software

1. Introducción

En el panorama actual del desarrollo de software, la complejidad y las demandas de escalabilidad, disponibilidad y rendimiento han llevado a la evolución de los estilos arquitectónicos. Ya no es suficiente con diseñar sistemas monolíticos; los arquitectos deben comprender y aplicar una variedad de estilos de arquitectura distribuida para satisfacer las necesidades del negocio moderno.

El presente seminario tiene como objetivo analizar cinco estilos arquitectónicos fundamentales en el contexto de los sistemas distribuidos. Estos estilos no solo representan diferentes formas de organizar el código y los componentes, sino que también encapsulan un conjunto único de *trade-offs*[1] (compensaciones) que impactan directamente en las características arquitectónicas del sistema, como la escalabilidad, la elasticidad, la tolerabilidad a fallos y la mantenibilidad.

La selección de estas cinco arquitecturas se debe a su relevancia actual en la industria y a que representan un espectro de soluciones que van desde lo monolítico hasta lo altamente distribuido, permitiendo un análisis comparativo profundo.

Service-Based Architecture (Arquitectura Basada en Servicios): Es un híbrido pragmático que combina la simplicidad de un monolito con la flexibilidad de los servicios distribuidos. Típicamente, consta de una interfaz de usuario separada, un pequeño número de servicios de dominio de grano grueso (generalmente de 4 a 12) desplegables independientemente y una base de datos compartida (o parcialmente compartida). Es una opción popular para aplicaciones empresariales que necesitan cierto nivel de modularidad sin la complejidad operativa total de los microservicios. **Nota:** Aunque comparte la base de datos, no es un monolito; los servicios se despliegan de forma independiente.

Event-Driven Architecture (Arquitectura Dirigida por Eventos): Se centra en la producción, detección, consumo y reacción a eventos. Es un estilo altamente escalable y adaptable, ideal para sistemas que requieren un alto rendimiento y una respuesta dinámica a situaciones cambiantes. Se divide en dos topologías principales: la topología de *broker* (descentralizada) y la topología de *mediator* (centralizada).

[1] En ingeniería de software, un *trade-off* o compensación es la decisión de sacrificar o disminuir una cualidad o atributo de un sistema (por ejemplo, el rendimiento) para obtener o mejorar otro (por ejemplo, la mantenibilidad).

Space-Based Architecture (Arquitectura Basada en Espacio): Diseñada para abordar problemas de alta escalabilidad, elasticidad y concurrencia extrema. Su característica principal es la eliminación de la base de datos central como cuello de botella, utilizando en su lugar una red de unidades de procesamiento con réplicas de datos en memoria (data grids). Esto permite una escalabilidad casi infinita y una elasticidad variable.

Service-Oriented Architecture (SOA): Un estilo arquitectónico histórico pero crucial para entender la evolución. Se caracteriza por servicios de grano fino y altamente reutilizables. **SOA admite tanto orquestación (con un motor central) como coreografía (descentralizada)**. Su filosofía se centró en la reutilización empresarial, pero a menudo resultó en un alto acoplamiento y una complejidad operativa significativa. La orquestación es solo una de sus variantes.

Microservices Architecture (Arquitectura de Microservicios): Es un estilo distribuido que lleva el concepto de *bounded context* del Diseño Dirigido por Dominio (DDD) a su máxima expresión. Cada servicio es pequeño, independiente, desplegable de forma autónoma y posee su propia base de datos. El objetivo principal es un alto desacoplamiento y la capacidad de evolución independiente de cada servicio, lo que permite una gran agilidad y escalabilidad.

2. Preguntas

Las siguientes preguntas están diseñadas para que los estudiantes apliquen los conceptos teóricos de la bibliografía a escenarios prácticos y realicen análisis comparativos profundos para resolver problemas de diseño arquitectónico.

1. **Análisis Comparativo de Comunicación y Acoplamiento:** Un sistema de subastas en línea requiere que, al finalizar una subasta, se realicen múltiples acciones: notificar al ganador, procesar el pago y actualizar el inventario. En un diseño con **Microservicios**, un arquitecto podría optar por una coreografía de eventos (comunicación asíncrona) o una orquestación mediante un servicio mediador. En un diseño con **Service-Based Architecture**, estas acciones probablemente se ejecutarían dentro de un mismo servicio de dominio de grano grueso. Basado en los conceptos de *connascence* (connascencia) y los *trade-offs* de la comunicación síncrona vs. asíncrona, analice:
   - ¿Cómo afecta cada enfoque (orquestación vs. coreografía en microservicios, y servicios de grano grueso en SBA) al acoplamiento dinámico entre los componentes del sistema?
   - ¿Qué implicaciones tiene cada diseño para la tolerancia a fallos y la consistencia de los datos (ACID vs. BASE) si el servicio de pago falla justo después de que se notifica al ganador?
2. **Diseño para Elasticidad Extrema y Consistencia de Datos:** Considere el sistema de venta de entradas para conciertos descrito en el libro (Capítulo 15). Durante el lanzamiento de entradas para un artista popular, el sistema experimenta un aumento repentino de usuarios de cientos a decenas de miles en segundos. Se le pide como arquitecto diseñar la solución.

- Justifique por qué una **Space-Based Architecture** sería una elección más adecuada que una **Microservices Architecture** para este escenario específico, enfocándose en las características de elasticidad y escalabilidad.
- Sin embargo, la Space-Based Architecture introduce el problema de las colisiones de datos en cachés replicadas. Describa cómo un arquitecto podría mitigar el riesgo de colisiones de datos en el contexto de la venta de asientos numerados, donde la consistencia es crucial. ¿Qué mecanismos o *trade-offs* consideraría?

3. **Análisis de Fallos y Resiliencia en Arquitecturas Dirigidas por Eventos:** En una **Event-Driven Architecture** con topología de *Broker*, un procesador de eventos (*Event Processor*) falla repetidamente al procesar un tipo específico de mensaje debido a un error en los datos.
   - Describa el impacto de este fallo en el flujo general del sistema y en la experiencia del usuario final, considerando que no hay un mediador central.
   - Proponga una solución arquitectónica utilizando el *Workflow Event Pattern* (Patrón de Eventos de Flujo de Trabajo) descrito en el Capítulo 14. Explique detalladamente cómo este patrón mitiga el problema y qué componentes adicionales (como el *workflow processor* y el *dashboard*) serían necesarios, así como los *trade-offs* que introduce (ej. procesamiento fuera de orden).
4. **El Dilema de la Granularidad y el Acoplamiento en Microservicios:** Un equipo de desarrollo está migrando un sistema monolítico de gestión de recursos humanos a **Microservicios**. Inicialmente, crean servicios muy pequeños para cada entidad (Empleado, Departamento, Nómina, Beneficios). Pronto descubren que completar una operación simple, como contratar a un nuevo empleado, requiere una compleja coreografía de llamadas síncronas entre casi todos los servicios, lo que resulta en un alto acoplamiento y una fragilidad similar a la del monolito original.
   - Basado en los principios de *bounded context* y las guías de granularidad del Capítulo 17, ¿cuál fue el error fundamental en el diseño de los servicios?
   - Proponga una nueva estrategia de descomposición. ¿Cómo aplicaría los conceptos de propósito, transacciones y coreografía para redefinir los límites de los servicios y reducir el acoplamiento? Proporcione un ejemplo de cómo agruparía los servicios para la operación de çontratación".
5. **Comparación de Atributos de Calidad y Trade-offs:** Un director de tecnología (CTO) le pide que elija un estilo arquitectónico para un nuevo sistema de procesamiento de transacciones financieras. Los requisitos clave son: 1) Consistencia de datos absoluta (ACID), 2) Capacidad de desplegar nuevas características de negocio de forma independiente y rápida, y 3) Un equipo de desarrollo con experiencia mixta.
   - Compare **Service-Based Architecture** y **Microservices Architecture** en función de estos tres requisitos. ¿Cuál de las dos arquitecturas es más adecuada para cumplir con el requisito de consistencia de datos y por qué?
   - ¿Cómo se ve afectada la capacidad de despliegue independiente (requisito 2) en cada estilo? ¿Qué *trade-offs* debe aceptar el CTO si prioriza la consistencia sobre la agilidad de despliegue?

6. **El Legado de SOA y su Influencia en Arquitecturas Modernas:** La **Service-Oriented Architecture (SOA)** con orquestación a menudo es criticada por su complejidad y por crear un cuello de botella central (el motor de orquestación). Sin embargo, algunos de sus principios, como la reutilización de servicios, siguen siendo relevantes. **SOA también admite coreografía, no solo orquestación.**

   - Analice cómo la **Microservices Architecture** aborda el problema de la reutilización de una manera fundamentalmente diferente. ¿Cómo maneja la reutilización de lógica de negocio (ej. validación de clientes) vs. la reutilización operativa (ej. logging, monitoreo)?
   - ¿En qué escenarios un arquitecto podría todavía considerar un enfoque de orquestación (como un mediador local en microservicios o un *orchestration engine* en SOA) a pesar de sus desventajas? Proporcione un ejemplo de un flujo de trabajo complejo que se beneficiaría de la orquestación explícita.

3. Orientaciones Generales

Para garantizar la correcta entrega y evaluación del seminario, todos los equipos deben cumplir con las siguientes indicaciones.

Formato y Presentación del Documento

El documento final del seminario debe cumplir con los siguientes requisitos de formato:

- **Página de presentación:** El documento debe incluir una página de presentación (portada) que contenga de forma clara y explícita la siguiente información:
  1. Nombre de la actividad: "Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio".
  2. Nombre del equipo.
  3. Nombres completos de todos los integrantes del equipo.
  4. Indicación explícita de la pregunta o preguntas en las que trabajó cada integrante. Es indispensable que se detalle la contribución individual de cada miembro.
  5. Enlace al repositorio público donde se encuentra alojado el código fuente del proyecto.
- **Nombre del fichero:** El archivo del documento debe ser nombrado siguiendo el formato `SemArq1_NombreEquipo`. Por ejemplo, si el equipo se llama "Los Arquitectos", el fichero debe llamarse `SemArq1_LosArquitectos.pdf` (o la extensión correspondiente).
- **Documento:** Cada equipo entregará un documento único con las respuestas a todas las preguntas planteadas en la sección anterior.

Uso de Fuentes y Citas Bibliográficas

Todo el contenido del documento debe ser original y estar debidamente fundamentado. En consecuencia, se establecen las siguientes normas de obligado cumplimiento:

- **Cita obligatoria de todas las fuentes:** Cualquier idea, concepto, definición, dato, figura o fragmento de texto que no sea de elaboración propia debe estar acompañado de su correspondiente cita bibliográfica en el cuerpo del documento. Esto aplica tanto a las obras listadas en la sección de *Bibliografía Recomendada* como a cualquier otra fuente externa (libros, artículos, sitios web, documentación técnica) que el equipo haya consultado y utilizado para la realización del seminario.
- **Formato de citas:** Se debe emplear un estilo de cita consistente a lo largo de todo el documento. Se recomienda el uso de un sistema de autor-año (por ejemplo, APA) o de notas al pie, siempre que se aplique de manera uniforme. Todas las fuentes citadas en el texto deben aparecer en la sección de referencias bibliográficas final, y viceversa.

Declaración de Uso de Inteligencia Artificial

En coherencia con los principios de transparencia y honestidad académica, se establece lo siguiente respecto al uso de herramientas de inteligencia artificial (IA):

- **Obligatoriedad de declaración:** Si el equipo ha utilizado alguna herramienta de inteligencia artificial (como ChatGPT, Gemini, Claude, Copilot, u otras similares) para la elaboración del seminario, es obligatorio incluir un epígrafe independiente titulado **"Declaración de Uso de la Inteligencia Artificial"** al final del documento, después de la bibliografía.
- **Contenido de la declaración:** En dicho epígrafe se debe explicar de forma clara y detallada:
  1. **Qué herramienta(s) de IA** fue(ron) utilizada(s), especificando su nombre y versión cuando sea posible.
  2. **Con qué propósito** fue(ron) empleada(s) (por ejemplo: generación de ideas, redacción de borradores, corrección de estilo, traducción, generación de código LaTeX, elaboración de diagramas).
  3. **De qué forma** se integró el resultado en el documento final (por ejemplo: se utilizó como base para una sección que luego fue reescrita y verificada por el equipo, se empleó únicamente para mejorar la redacción, se usó para generar ejemplos de código que fueron analizados y adaptados, etc.).
- **Responsabilidad del contenido:** El uso de IA no exime al equipo de la responsabilidad sobre la totalidad del contenido entregado. Los estudiantes son los únicos responsables de verificar la veracidad, originalidad y calidad de la información generada, así como de asegurarse de que no se incurra en plagio o en el uso indebido de fuentes.
- **Ejemplo de epígrafe:** A modo de ilustración, la declaración podría tener la siguiente estructura:

Declaración de Uso de la Inteligencia Artificial
Para la elaboración de este seminario, el equipo utilizó la herramienta ChatGPT (versión GPT-4) con el propósito de generar ideas iniciales para la estructura del documento y mejorar la redacción de algunos párrafos. Los integrantes que hicieron uso de esta herramienta fueron [Nombre 1] y [Nombre 2]. Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.

Entrega y Repositorio

La entrega del seminario se realizará a través de dos canales obligatorios y complementarios:

- **Plataforma de evaluaciones:** El documento final debe ser subido a la sección "Evaluaciones" del grupo correspondiente a la asignatura **Ingeniería de Software** en Telegram.
- **Repositorio del proyecto:** El mismo documento, junto con cualquier material adicional (código fuente en LaTeX, diagramas, ADRs, matrices de trade-offs o cualquier material complementario), debe estar alojado en el repositorio del proyecto del equipo. El enlace a dicho repositorio debe ser incluido en la página de presentación del documento, tal como se indicó anteriormente.

4. Bibliografía Recomendada

Para el desarrollo de este seminario, se utilizará como referencia principal el siguiente texto, el cual cubre en profundidad los estilos arquitectónicos analizados. Se indican los capítulos específicos que deben ser consultados para cada tema.

- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.
  - **Para la introducción a los estilos arquitectónicos y la comparación monolito vs. distribuido:** Capítulos 9 y 18.
  - **Para Service-Based Architecture:** Capítulo 13.
  - **Para Event-Driven Architecture:** Capítulo 14.
  - **Para Space-Based Architecture:** Capítulo 15.
  - **Para Service-Oriented Architecture (Orquestación y Coreografía):** Capítulo 16.
  - **Para Microservices Architecture:** Capítulo 17.
  - **Para conceptos de acoplamiento, connascencia y arquitectura quantum:** Capítulos 3, 7 y 8.
- Ford, N., Richards, M., Sadalage, P., & Dehghani, Z. (2021). *Software Architecture: The Hard Parts*. O'Reilly Media.
  - **Para datos distribuidos, consistencia eventual y patrones como Saga, CQRS y Event Sourcing:** Capítulos 4, 6, 7 y 9.

- Ford, N., Parsons, R., & Kua, P. (2017). *Building Evolutionary Architectures: Automated Software Governance*. O'Reilly Media.
    - **Para fitness functions, evolución guiada y gobierno arquitectónico:** Capítulos 2, 3 y 5.